

DistroNexus Template System Comprehensive Guide

Last Updated: 2026-02-15

Owner: Copilot (GPT-5.3-Codex)

1. Purpose

This document provides an end-to-end view of the DistroNexus template system, including:

- requirement analysis,
- current implementation architecture,
- built-in template catalog,
- and contributor/developer guidance.

It is intended to serve as the single working reference for product, engineering, QA, and documentation updates related to templates.

2. Requirement Analysis

2.1 Problem Statement

WSL setup is often repetitive and inconsistent across developers and machines. The template system addresses this by providing declarative metadata + executable scripts to bootstrap repeatable environments.

2.2 Requirement Sources

Primary requirement inputs in this repository:

- `docs/specs/template-system-requirements.md`
- `docs/specs/built-in-template-expansion-requirements.md`
- `docs/specs/built-in-template-automation-test-suite-requirements.md`
- `docs/development/template-expansion-implementation-task-list.md`
- `docs/development/built-in-template-automation-implementation-task-list.md`

2.3 Functional Requirements (Consolidated)

The template system must provide:

1. **Catalog-based discovery**

- Load templates from `config/templates.json`.
- Support search/filter by ID/category/tags.

2. **Scripted application workflow**

- Apply a template to a target WSL instance.
- Support script ordering, timeout control, and continue-on-error semantics.

3. **Version-aware templates**

- Support **VersionOptions** and **DefaultSelections** for SDK/channel selection.

4. Preflight checks and guarded scenarios

- Execute preflight checks before template scripts.
- Enforce required checks and report actionable errors.

5. History and observability

- Persist template application history under AppData.
- Expose progress and output for long-running operations.

6. Automation test runner support

- Validate all templates or selected templates through **Invoke-DistroNexusTemplateAutomation**.
- Support result classification (**Pass** / **Fail** / **Blocked**) and artifact output.

2.4 Non-Functional Requirements (Consolidated)

- **Idempotency-first:** scripts should be safe to rerun.
- **Safety:** no absolute script paths; block path traversal.
- **Local-first validation:** full automation runs on local Windows + WSL2.
- **Compatibility awareness:** templates declare compatible distros and scenario tags.
- **Traceability:** store machine-readable and markdown run artifacts for automation runs.

2.5 Current Alignment Notes

- Implemented categories include: **Development**, **Platform**, **CloudNative**, **Database**, **DataAndAI**.
- The expansion spec also mentions **DevOps** as a category target; a dedicated built-in **DevOps** category template is not currently present.
- Mandatory Phase-1 breadth is achieved with 15 built-in templates in the current catalog.

3. Implementation Overview

3.1 Architecture Layers

1. Metadata layer

- **config/templates.json** defines template identity, options, checks, scripts, and artifacts.

2. Core domain layer

- **src/Client/DistroNexus.Core/Models/Template.cs**
- Contains template model, script model, version options, preflight checks, output artifacts, and result/progress models.

3. Service layer

- **src/Client/DistroNexus.Core/Interfaces/ITemplateService.cs**
- **src/Client/DistroNexus.Core/Services/TemplateService.cs**

- Handles loading, search, validation, apply execution, compatibility checks, import/export, and history persistence.

4. Desktop workflow integration

- `src/Client/DistroNexus.Desktop/Wizard/InstallWizardWorkflowViewModel.cs`
- `SelectTemplateStep` for selection.
- `TemplateApplyStep` for execution and progress reporting.

5. PowerShell command surface

- `src/PowerShell/Public/Get-DistroNexusTemplate.ps1`
- `src/PowerShell/Public/Apply-DistroNexusTemplate.ps1`
- `src/PowerShell/Public/Invoke-DistroNexusTemplateAutomation.ps1`
- Exported in `src/PowerShell/DistroNexus.psd1`.

3.2 TemplateService Execution Flow

`TemplateService.ApplyTemplateAsync(...)` high-level flow:

1. Resolve template by ID.
2. Merge effective variables (`Variables`, `DefaultSelections`, `VersionOptions.DefaultValue`, runtime overrides).
3. Run preflight checks (required checks fail-fast).
4. Execute scripts in `Order` sequence.
5. For Bash scripts, execute inside WSL target instance.
6. Report `TemplateProgress` during execution.
7. Persist `TemplateApplicationRecord` history.

3.3 Security and Safety Controls

Current implementation enforces:

- Rejection of absolute script paths.
- Path traversal protection via root validation.
- Controlled script resolution under allowed roots.
- Cancellation support and timeout wiring.
- Explicit handling for `ContinueOnError`.

3.4 Persistence and Runtime Paths

Primary template-related files under AppData:

- `%APPDATA%\DistroNexus\templates.json` (user/custom template cache)
- `%APPDATA%\DistroNexus\template-application-history.json` (history)

Built-in catalog source:

- `config/templates.json`

4. Built-in Template Catalog

Current built-in templates (from `config/templates.json`):

Category	Template ID	Name	Description
CloudNative	<code>container-runtime-dev</code>	Container Runtime Development	Configure container runtime mode for WSL2 development workflows.
CloudNative	<code>kubernetes-local-dev</code>	Kubernetes Local Development	Install local Kubernetes tooling with selectable cluster mode.
DataAndAI	<code>ai-ml-gpu-dev</code>	AI/ML GPU Development	Set up AI/ML environment with CPU or GPU-oriented profile and fallback guidance.
Database	<code>database-local-stack</code>	Database Local Stack	Install selectable local database services and tooling in WSL2.
Development	<code>docker-dev</code>	Docker Development	Install Docker Engine and Docker Compose plugin.
Development	<code>dotnet-dev</code>	.NET Development	Install .NET SDK and essential .NET development tools for Ubuntu and Debian.
Development	<code>dotnet-multi-sdk-dev</code>	.NET Multi-SDK Development	Install .NET SDK with channel-based version selection and optional <code>global.json</code> generation.
Development	<code>fullstack-dev</code>	Fullstack Development	Install .NET, Node.js, Python, and Docker in one template.
Development	<code>go-dev</code>	Go Development	Install Go stable toolchain with version pin support.
Development	<code>nodejs-dev</code>	Node.js Development	Install Node.js LTS with npm, yarn, and pnpm.
Development	<code>nodejs-multi-version-dev</code>	Node.js Multi-Version Development	Install Node.js via nvm with selectable channel and optional <code>.nvmrc</code> generation.
Development	<code>python-dev</code>	Python Development	Install Python 3, pip, venv, and Poetry for Python development.
Development	<code>python-multi-version-dev</code>	Python Multi-Version Development	Install Python via pyenv with optional project pinning and environment tool bootstrap.
Development	<code>rust-dev</code>	Rust Development	Install Rust with rustup channel selection.
Platform	<code>java-jvm-dev</code>	Java/JVM Development	Install Java SDK with SDKMAN and optional <code>.sdkmanrc</code> generation.

5. Developer Guide

5.1 Add a New Built-in Template

1. Add metadata to `config/templates.json`:
 - `Id, Name, Category, Description, CompatibleDistros, ScenarioTags`
 - `InstallMode, VersionOptions, DefaultSelections`
 - `PreflightChecks, OutputArtifacts, Scripts`
2. Add script assets under `config/templates/<template-id>/`.
3. Ensure script paths in metadata are relative paths under `config/templates/`.
4. Validate model compatibility with:
 - `TemplateService.ValidateTemplateAsync(...)`
 - and local template application smoke test.

5.2 Script Authoring Guidelines

- Make scripts idempotent (safe to run repeatedly).
- Fail fast with actionable messages.
- Support variable placeholders (`${VARIABLE_NAME}`) where needed.
- Avoid hardcoded machine-specific paths.
- Keep one responsibility per script step and use ordering for composition.

5.3 PowerShell Usage (Developer/Operator)

Discover templates:

```
Get-DistroNexusTemplate
Get-DistroNexusTemplate -Category Development
Get-DistroNexusTemplate -Id dotnet-multi-sdk-dev
```

Apply template:

```
Apply-DistroNexusTemplate -InstanceName Ubuntu-22.04 -TemplateId python-dev -
Verbose
Apply-DistroNexusTemplate -InstanceName Ubuntu-22.04 -TemplateId nodejs-dev -
Variables @{ SDK_NODE_CHANNEL = 'lts/*' }
```

Run automation validation:

```
Invoke-DistroNexusTemplateAutomation -Mode AllTemplates -Distro Ubuntu-22.04
Invoke-DistroNexusTemplateAutomation -Mode SelectedTemplates -TemplateIds 'dotnet-
dev','nodejs-dev' -Distro Ubuntu-22.04 -DryRun
```

5.4 Testing and Verification Path

Recommended developer validation sequence:

1. **Metadata sanity**

- Validate JSON shape and required fields.

2. **Single-template local run**

- Apply target template to a controlled distro instance.

3. **Selective automation**

- `Invoke-DistroNexusTemplateAutomation -Mode SelectedTemplates ...`

4. **Full catalog regression (when needed)**

- `Invoke-DistroNexusTemplateAutomation -Mode AllTemplates ...`

5. **Artifact review**

- Check output under `docs/development/testing/results/`.

5.5 Common Pitfalls

- Using absolute `ScriptPath` values (blocked by service path guard).
- Missing defaults for required `VersionOptions`.
- Omitting scenario preflight checks for capability-gated templates.
- Creating scripts that assume one distro flavor without `CompatibleDistros` constraints.

6. Maintenance Checklist

When updating the template system:

- Update this guide if data model or execution behavior changes.
- Keep template catalog and docs synchronized.
- Keep release notes and website docs aligned with template scope.
- Preserve bilingual website communication for template-facing changes.

7. Recommended Next Improvements

1. Add explicit DevOps-category built-in templates to close category parity.
2. Add schema validation tooling for `config/templates.json` in pre-merge checks.
3. Add lightweight compatibility matrix docs per template and distro.
4. Add reusable script helper library under `config/templates/common/` and document conventions.